

Technical Execution Plan: Online Scene Change Detection (O-SCD)

Custom Dataset Generation and Training Pipeline

Joshua Hecke

September 7, 2026

1 Introduction

This document outlines the end-to-end technical steps for creating a custom time-variant dataset and training the **O-SCD** (Online Scene Change Detection) framework. Because we do not yet have a multi-temporal underwater dataset, we must construct a custom dataset consisting of two states: a reference scene (Object A) and an inference/query scene (Object A + Object B).

O-SCD operates in a label-free, online manner by relying on a pre-computed 3D Gaussian Splatting (3DGS) reconstruction of the *reference scene*, and performs PnP-based pose estimation on the fly for the *inference scene*.

2 Phase 1: Dataset Generation and Structure

To successfully run O-SCD, your custom dataset must adhere exactly to the directory structure expected by the dataloaders.

2.1 Image Capture

1. **Reference Scene (Time 1):** Capture a set of images (e.g., 50-150) covering the target object/area (Object A). Save these images into a folder named `input` inside a `reference_scene` directory.
2. **Inference Scene (Time 2):** Introduce the change (Object B) into the same environment. Capture a new set of images from varying perspectives. Save these images directly into an `images` folder inside an `inference_scene` directory.

2.2 Expected Directory Layout

Before running any scripts, organize your custom dataset as follows:

```
1 ~/UMP/Codebase/Dataset/Custom_OSCD_Dataset/  
2 |--- reference_scene/  
3 |   |--- input/  
4 |       |--- frame001.jpg  
5 |       |--- frame002.jpg ...  
6 |--- inference_scene/  
7 |   |--- input/  
8 |       |--- frame001.jpg  
9 |       |--- frame002.jpg ...
```

3 Phase 2: Preparation and COLMAP Processing

CRITICAL PREPROCESSING STEP: O-SCD requires camera poses **only** for the reference scene (poses for the inference scene are estimated on the fly via PnP). However, because O-SCD directly inherits the *undistorted* pinhole camera intrinsics from the reference scene and applies them to the inference scene, **the inference images must also be undistorted** to prevent systematic projection errors.

The easiest way to achieve this is to run the standard 3DGS `convert.py` on **both** scenes independently. This will generate undistorted images in an `images/` folder for both.

```

1 # Use the standard 3DGS conversion tool to run COLMAP
2 cd ~/UMP/Codebase/Tools/gaussian-splatting-main
3 conda activate colmap_runner
4
5 # Generate sparse point cloud and undistorted images for the reference scene
6 python convert.py -s ~/UMP/Codebase/Dataset/Custom_OSCD_Dataset/reference_scene
7
8 # Generate undistorted images for the inference scene (O-SCD will ignore its sparse
   poses)
9 python convert.py -s ~/UMP/Codebase/Dataset/Custom_OSCD_Dataset/inference_scene

```

This will create `sparse/0` and `images/` inside both directories. O-SCD's dataloader is hardcoded to load from `inference_scene/images`, which will now correctly contain undistorted images.

4 Phase 3: Reference Scene Reconstruction

O-SCD requires a fully trained 3DGS point cloud of the reference scene as a prior. It expects this reconstruction to be stored in a directory named `reference_reconstruction` at the root of the dataset.

```

1 # Switch to the standard 3DGS environment
2 conda activate 3dgs # (or whichever env you use for standard 3DGS)
3
4 # Train the baseline model on the reference scene
5 # By setting -m to reference_reconstruction, it saves directly where O-SCD expects it
6 python train.py -s ~/UMP/Codebase/Dataset/Custom_OSCD_Dataset/reference_scene \
7                 -m ~/UMP/Codebase/Dataset/Custom_OSCD_Dataset/reference_reconstruction

```

Verify that `~/UMP/Codebase/Dataset/Custom_OSCD_Dataset/reference_reconstruction/point_cloud/iteration_30` exists.

5 Phase 4: Running O-SCD

With the environment prepared, you can now run the O-SCD pipeline.

5.1 Environment Setup

```

1 cd ~/UMP/Codebase/3DGS-Change-Detection/O-SCD-main
2
3 # Initialize and update submodules (CRITICAL for requirements.txt to succeed)
4 git submodule update --init --recursive
5
6 conda create -n oscd python=3.12 -y
7 conda activate oscd
8
9 # Install dependencies (CUDA 12.x recommended based on RTX 5070 Ti)
10 pip install torch torchvision xformers --index-url https://download.pytorch.org/whl/
   cu121
11 pip install cupy-cuda12x
12 pip install -r requirements.txt

```

5.2 Executing Online Change Detection

Run the core `oscd.py` script. This script loads the reference point cloud, aligns the inference images, detects the changes, and generates the change masks.

```

1 # Run Change Detection
2 # --test_hold 5 holds out 1/5th of the frames for testing (if needed)
3 # --resolution 1 ensures full resolution (default scripts use 4 for downsampling)
4 python oscd.py -s ~/UMP/Codebase/Dataset/Custom_OSCD_Dataset \
5               -m ~/UMP/Codebase/Dataset/Custom_OSCD_Dataset/output \
6               --resolution 1 \
7               --test_hold 5

```

5.3 Executing Scene Update (Optional)

O-SCD also provides an update script to modify the 3D Gaussian Splatting scene representation based on the detected changes.

```
1 python update.py -s ~/UMP/Codebase/Dataset/Custom_OSCD_Dataset \  
2                  -m ~/UMP/Codebase/Dataset/Custom_OSCD_Dataset/output \  
3                  --resolution 1
```